TriageFlow ML-Engine: microservicio de triaje con Machine Learning

Laboratorio 4 - Contenedores y Despliegue con Docker

Universidad de Antioquia - Cloud Computing

Samuel Puerta Patiño - CC 1023624795

https://github.com/SamuelPuerta/triageflow-ml-service & **DockerHub Image: samuelpuerta/triageflow-ml**

1. Descripción

TriageFlow ML-Engine es un microservicio HTTP del dominio HealthTech/MLOps que clasifica a pacientes de urgencias en tres niveles de triaje (1 crítico, 2 urgente, 3 estable) a partir de sus signos vitales, y calcula indicadores de saturación sin consultar bases de datos externas. Está escrito en Python 3.14 con FastAPI y Uvicorn; la inferencia usa un RandomForestClassifier de scikit-learn, entrenado offline y serializado con Joblib. El modelo se carga una sola vez al iniciar el contenedor y cada evento se registra en un buffer en memoria desde el que se calculan los KPIs.

Endpoint	Función
GET /health	Estado del servicio y disponibilidad del modelo
POST /api/v1/events/patient-checkin	Recibe signos vitales, infiere el nivel de triaje (1 a 3) y su confianza
GET /api/v1/bi/er-status	KPIs agregados de saturación de urgencias en tiempo real

Contenerización. El Dockerfile es multi-stage sobre python:3.14-slim: una etapa instala las dependencias y la etapa final copia solo las librerías y el código, sin cachés ni herramientas de compilación. El contenedor corre con un usuario sin privilegios, expone el puerto 8000 e incluye un HEALTHCHECK sobre /health. La imagen se construye con docker build, se ejecuta con docker run -p 8000:8000 y se publica en Docker Hub (docker.io/[usuario]/triageflow-ml:v1). El modelo se entrenó con datos clínicos simulados, por lo que el proyecto es académico y no una herramienta de decisión médica.

2. Aporte de Docker a la arquitectura de despliegue

- **Portabilidad.** Las librerías binarias (NumPy, scikit-learn) viajan dentro de la imagen, de modo que el servicio corre igual en cualquier equipo con Docker, sin depender del sistema operativo ni de la versión de Python instalada.
- **Aislamiento.** El servicio y sus dependencias están separados del host y de otras aplicaciones, lo que evita conflictos de versiones; además corre sin privilegios de root.
- **Reproducibilidad.** Con versiones exactas en requirements.txt y una imagen base fija, la inferencia se ejecuta igual en el entorno local y en producción. Un archivo .joblib puede comportarse distinto si se carga con otra versión de scikit-learn, y la imagen lo evita.
- **Inmutabilidad.** El modelo entrenado queda embebido en la imagen y el script de entrenamiento no se incluye. Actualizar el modelo implica construir y publicar una nueva versión etiquetada, lo que deja trazabilidad y permite volver a una versión anterior.
- **Escalabilidad.** Al ser una imagen autocontenida y sin estado de configuración externo, se pueden lanzar varias réplicas idénticas detrás de un balanceador con un orquestador.

3. Arquitectura propuesta en AWS

La imagen se publicaría en Amazon ECR y se ejecutaría como servicio de Amazon ECS con Fargate, sin administrar servidores. Un Application Load Balancer repartiría el tráfico entre las tareas y usaría /health como verificación de salud. Los eventos de admisión se recibirían a través de Amazon EventBridge, que invocaría el endpoint de ingesta, y las métricas se exportarían a Amazon QuickSight para los tableros de BI. El autoescalado de tareas se configuraría según CPU o número de peticiones.

Limitación a resolver. Los KPIs viven en la memoria de cada contenedor, así que con varias réplicas cada una tendría métricas parciales. En producción el estado se externalizaría a un almacén compartido (por ejemplo DynamoDB o Redis) o los eventos se enviarían a un pipeline de datos hacia QuickSight.

4. Video Explicación: https://drive.google.com/file/d/1pzkMabtRYkA19GGufcc7ci7JAroRgiBs/view?usp=sharing